Maximaux, multiplexés, compressés et fractals

OPTION INFORMATIQUE - Devoir n° 4 - Olivier Reynet

Toutes les réponses informatiques à cet examen sont à fournir en langage OCaml.

A Sous-tableau de somme maximale

Le problème du sous-tableau de somme maximale s'applique à un tableau d'entiers. il s'agit de trouver le sous-tableau dont la somme des éléments est la plus grande.

■ **Définition 1 — Sous-tableau.** Un sous-tableau d'un tableau est un tableau d'éléments **contigus** contenu dans le tableau.

■ **Exemple 1 — Calcul du sous-tableau à somme maximale.** Soit le tableau d'entiers `[|-2 ;-3 ;4 ;-1 ;-2 ;1 ;5 ;-3|]`.
Le sous-tableau présentant la somme maximale est `[|4 ;-1 ;-2 ;1 ;5|]` et la somme vaut 7.

(R) Lorsque les éléments du tableau sont tous positifs ou nuls, le problème est trivial : il suffit de faire la somme des éléments. Lorsque les éléments sont tous négatifs ou nuls, il suffit de renvoyer l'élément le plus grand. On ne considèrera donc pas ces cas par la suite. Le tableau considéré contiendra toujours des éléments positifs et négatifs.

(R) Savoir résouvre ce problème est très important en traitement de signal : il s'agit de détecter des zones remarquables dans des signaux (des points brillants dans une image par exemple), en médecine ou dans le domaine de l'imagerie radar/sonar.

Les trois sous-sections qui suivent permettent de programmer l'algorithme du sous-tableau de somme maximale de trois approches différentes :

- une approche naïve, par la force brute ou par exhaustion,
- une approche diviser pour régner,
- et une approche gloutonne.

Ces trois sous-sections sont indépendantes.

(R) La programmation OCaml de cette section est principalement de nature **impérative** et a pour but de tester votre capacité à utiliser les tableaux, les références et les boucles `for`.

a Approche par force brute

A1. Écrire une fonction de signature `local_sum : int array -> int -> int -> int` qui prend en paramètre un tableau et deux indices `start` et `stop` et qui renvoie la somme locale associée au sous-tableau délimité par les indices `start` et `stop`. On supposera que `0 <= start <= stop < n`, si `n` est la

dimension du tableau. Par exemple, `local_sum [|2 ; −4 ; 1 ; 9 ; −6 ; 7 ; −3|] 2 4` renvoie 4 car il s'agit de la somme de `[|1 ;9 ;−6|]`.

Solution :

```
let local_sum t start stop =
    let s = ref 0 in
    for k = start to stop do
         s := !s + t.(k);
    done;
    !s;;
```

A2. Quelle est la complexité de la fonction `local_sum`?

Solution : Elle est en O(stop-start+1), car l'instruction de la boucle (de complexité constante) est répétée stop−start+1 fois.

A3. En calculant toutes les sommes partielles de tous les sous-tableaux possibles, écrire une fonction de signature `brute_max_sum : int array −> int * int * int` qui renvoie un tuple d'entiers constitué de la somme maximale et des indices de début et de fin du sous-tableau correspondant à cette somme maximale. Par exemple : `brute_max_sum [|−2 ;−3 ;4 ;−1 ;−2 ;1 ;5 ;−3|]` renvoie : `(7,2,6)`.

Solution :

```
let brute_max_sum t =
    let n = Array.length t
    and max_sum = ref (−max_int)
    and start = ref 0 and stop = ref 0 in
    for i = 0 to n − 1 do
        for j = i to n − 1 do
            let loc_s = local_sum t i j in
            if loc_s > !max_sum
            then (max_sum := loc_s; start := i; stop := j)
        done;
    done;
    (!max_sum,!start,!stop);;
```

A4. Quelle est la complexité de la fonction `brute_max_sum`?

Solution : $C(n) = \sum_{i=0}^{n-1}\sum_{j=i}^{n-1} 1 = \sum_{i=0}^{n-1}(n-1-i) = \frac{n(n-1)}{2} = O(n^2)$ lorsqu'on programme **la seconde version**, sans utiliser la fonction `local_sum`.

Sinon, en utilisant la fonction `local_sum` .

$$C(n) = \sum_{i=0}^{n-1}\sum_{j=i}^{n-1} j-i+1 = \sum_{i=0}^{n-1}\sum_{k=0}^{n-1-i} k+1 = \sum_{i=0}^{n-1}(n-i) + \frac{(n-i-1)(n-i)}{2} = O(n^3)$$

En effet, le dernier terme de la somme engendre un terme $\sum_{i=0}^{n-1} i^2$ qui entraîne un n^3.

b Diviser pour régner

On se propose d'adopter une stratégie diviser pour régner afin d'améliorer la complexité :

1. Si le tableau est vide, renvoyer l'entier négatif le plus petit (en OCaml `-max_int`),
2. Si le tableau ne comporte qu'un seul élément, renvoyer cet élément,
3. Sinon diviser le tableau en deux moitiés et renvoyer le maximum de :
   (a) la solution du problème pour le tableau de gauche **sans** l'élément milieu (appel récursif),
   (b) la solution du problème pour le tableau de droite **sans** l'élément milieu (appel récursif),
   (c) la solution du problème lorsque le sous-tableau de somme maximale comprend l'élément milieu, c'est-à-dire lorsqu'il est **à cheval** sur le tableau droit et le tableau de gauche.

Dans un premier temps, on cherche à coder la sous fonction linéaire `cross_max_sum` qui trouve la solution du problème lorsque le sous-tableau de somme maximale est à cheval sur le tableau de gauche et de droite. Dans ce cas, la solution est forcément un sous-tableau qui contient l'élément du milieu. Il suffit donc de calculer :

- la somme maximale en partant du milieu en allant vers la gauche du tableau,
- la somme maximale en partant du milieu en allant vers la droite du tableau,

et de renvoyer le maximum de :

1. la somme à gauche,
2. la somme à droite,
3. ou la somme de droite et de gauche moins la valeur de l'élément milieu qui a été compté deux fois, une fois à droite, une fois à gauche.

R Afin de ne pas dupliquer en mémoire le tableau et les subdivisons du tableau, on repère par deux indices `start` et `stop` la partie du tableau sur laquelle les opérations s'effectuent. Ces deux indices sont passés en paramètres, en même temps que le tableau, aux fonctions qui résolvent le problème.

A5. Écrire une fonction de signature `cross_max_sum : int array -> int -> int -> int` dont les paramètres d'entrée sont :

- le tableau dont on cherche le sous-tableau de somme maximale,
- l'indice `start` du début du tableau considéré,
- l'indice `stop` de la fin du tableau considéré.

et qui renvoie, comme expliqué ci-dessus, la somme maximale à cheval sur les sous tableaux gauche (de `start` à `milieu`) et droit (de `milieu` à `stop`), où `milieu = (start + stop)/2`. Cette fonction est de complexité linéaire en fonction de `(stop - start + 1)`.
Par exemple : `cross_max_sum [|2 ; -4 ; 1 ; 9 ; -6 ; 7 ; -3|] 2 4` renvoie 10.

Solution :

```
let cross_max_sum t start stop =
    let middle = (start + stop) / 2 in
        let msg = ref t.(middle)
        and msd = ref t.(middle)
        and l_sum = ref t.(middle) in
        for k = 1 to (middle - start) do
            l_sum := !l_sum + t.(middle - k);
            if  !l_sum > !msg
            then  msg := !l_sum
        done;
        l_sum := t.(middle);
        for k = 1 to (stop - middle) do
            l_sum := !l_sum + t.(middle + k);
            if  !l_sum > !msd
            then msd := !l_sum
        done;
    let lm = !msg + !msd - t.(middle) in
    max lm (max !msd  !msg);;
```

A6. Écrire une fonction de signature `dac_max_sum : int array -> int` dont le paramètre d'entrée est le tableau dont on cherche le sous-tableau de somme maximale et qui renvoie la somme maximale trouvée. Cette fonction s'appuie sur une **fonction auxiliaire récursive** et implémente l'algorithme diviser pour régner décrit plus haut. Un code de départ est donnée :

```
let dac_max_sum t =
    let rec aux i j =
        ... (* à compléter *)
    in aux 0 (Array.length t - 1);;
```

Solution :

```
let dac_max_sum t =
    let rec aux i j =
    match j - i with
        | k when k < 0 ->(-max_int)
        | k when k = 0 -> t.(i)
        | _ -> let m = (i + j) / 2 in
                    let mtg = aux i (m - 1)
                    and mtd = aux (m + 1) j
                    and mcross = cross_max_sum t i j in
                    max (max mtg mtd) mcross
    in aux 0 (Array.length t - 1);;
```

A7. Quelle est la complexité de la fonction `dac_max_sum`?

Solution : En dehors des appels récursifs, la fonction `dac_max_sum` est de complexité linéaire à

cause de l'appel à la fonction `cross_max_sum`. On peut donc écrire, en notant n la taille du tableau :

$$T(n) = n + 2T((n-1)/2) \leqslant n + 2T((n)/2)$$

On reconnaît la récurrence du tri fusion. La complexité est donc en $\mathcal{O}(n \log n)$.

c Approche gloutonne

L'approche gloutonne[1] procède comme décrite par l'algorithme 1.

Algorithme 1 Approche gloutonne

```
Fonction KADANE(t)
    msum =← −∞
    lsum ← 0
    start ← 0
    stop ← 0
    s ← 0
    n ← taillle de t
    pour i de 0 à n - 1 répéter
        lsum ← lsum + t[i]
        si msum < lsum alors
            msum ← lsum
            start ← s
            stop ← i
        si lsum < 0 alors
            lsum ← 0
            s ← i + 1
    renvoyer (msum, start, stop)
```

A8. Expliquer comment fonctionne cet algorithme en l'appliquant au tableau `[|1 ;3 ;−5 ;−3 ;4 ;−1 ;−2 ;1 ;5 ;−3|]`. Décrire chaque itération de l'algorithme en faisant un tableau des valeurs des variables internes de l'algorithme : chaque itération sera une ligne de ce tableau.

Solution : L'algorithme calcule les sommes partielles et mémorise les sommes positives maximales. Dès qu'une somme partielle est négative, il l'exclue et continue en repartant de 0 à l'indice suivant. En effet, le prochain nombre positif sera dans tous les cas une somme plus importante que les sommes non maximales et négatives précédentes.

```
#i =   ⟶ msum lsum start stop s
#i = 0 ⟶ 1, 1, 0, 0, 0
#i = 1 ⟶ 4, 4, 0, 1, 0
#i = 2 ⟶ 4, 0, 0, 1, 3
#i = 3 ⟶ 4, 0, 0, 1, 4
#i = 4 ⟶ 4, 4, 0, 1, 4
```

1. algorithme de Kadane

```
#i = 5 ---> 4, 3, 0, 1, 4
#i = 6 ---> 4, 1, 0, 1, 4
#i = 7 ---> 4, 2, 0, 1, 4
#i = 8 ---> 7, 7, 4, 8, 4
#i = 9 ---> 7, 4, 4, 8, 4
- : int * int * int = (7, 4, 8)
```

A9. Quelle est la complexité de cet algorithme ?

Solution : La complexité de l'algorithme de Kadane est linéaire en fonction de la taille du tableau (n) car il effectue n itérations et les opérations dans la boucle sont de complexité $O(1)$.

A10. Écrire une fonction de signature `kadane : int array -> int * int * int` qui implémente l'algorithme 1.

Solution :

```
let kadane t =
    let m_sum = ref (-max_int)
    and l_sum = ref 0
    and start = ref 0
    and stop = ref 0
    and s = ref 0 in
    for i = 0 to (Array.length t - 1) do
        l_sum := !l_sum + t.(i);
        if !m_sum < !l_sum
        then (m_sum := !l_sum; start := !s; stop := i);
        if !l_sum < 0
        then (l_sum := 0; s := i + 1)
    done;
    (!m_sum, !start, !stop);;
```

B Multiplexeurs (d'après Mines Ponts 2006)

Soit $\mathbb{B} = (\{0,1\}, \neg, \vee, \wedge)$ l'algèbre de Boole, c'est-à-dire l'ensemble des valeurs vérités $\mathscr{B} = \{0,1\}$ qui représente le faux et le vrai muni des opérateurs négation, disjonction et conjonction logique.
Les portes logiques élémentaires sont les circuits électroniques à la base de tous les circuits complexes. Elles correspondent aux opérateurs $\neg, \vee$ et $\wedge$ de la logique. Ces portes sont représentées sur la figure 1.

■ **Définition 2 — Fonction logique.** Une fonction logique f à n variables est une application de $\mathscr{B}^n \to \mathscr{B}$.

B11. En utilisant deux portes logiques, créer un circuit électronique qui réalise la fonction constante $f : x \to 1$, c'est-à-dire la fonction constante $u : \mathscr{B} \to \mathscr{B}$ qui à toute variable logique fait correspondre la valeur de vérité 1 (vrai).Ce circuit possède une seule entrée et une seule sortie. Il est possible de dupliquer un signal en dédoublant un fil.

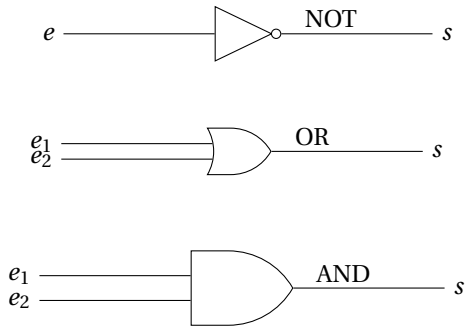
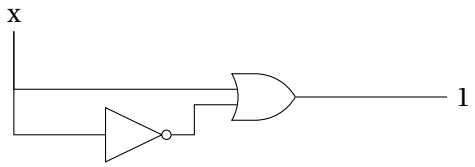


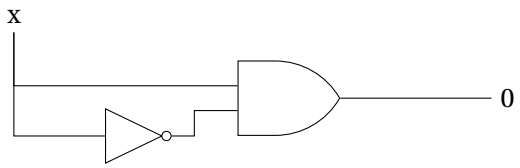
FIGURE 1 – Symboles des portes logiques élémentaires NOT, OR et AND

Solution : On cherche un circuit pour la formule logique $x \vee \neg x$ qui est une tautologie.



B12. En utilisant deux portes logiques, créer un circuit électronique qui réalise la fonction constante $f : x \to 0$, c'est-à-dire la fonction constante $z : \mathscr{B} \to \mathscr{B}$ qui à toute variable logique fait correspondre la valeur de vérité 0 (faux). Ce circuit possède une seule entrée et une seule sortie. Il est possible de dupliquer un signal en dédoublant un fil.

Solution : On cherche un circuit pour la formule logique $x \wedge \neg x$ qui est une antilogie. On peut obtenir ce résultat pas la loi de Morgan.



■ **Définition 3 — Multiplexeur.** Un multiplexeur (MUX) M_n est un circuit logique à $2^n - 1$ entrées de données d_i, $n-1$ entrées de sélection s_j et une sortie r. Il permet de sélectionner une entrée et de la recopier sur la sortie. La figure 2 représente un multiplexeur générique M_n.

Si on pose $k = \sum_{j=0}^{n-1} 2^j s_j$, alors le fonctionnement du multiplexeur garantit la recopie du bit d_k sur la sortie r.

B13. On considère le multiplexeur de la figure 3 obtenu pour $n = 1$. Exprimer la formule logique de la sortie du multiplexeur, c'est-à-dire exprimer r en fonction de d_0, d_1 et s_0.

Solution : $r = (\neg s_0 \wedge d_0) \vee (s_0 \wedge d_1)$

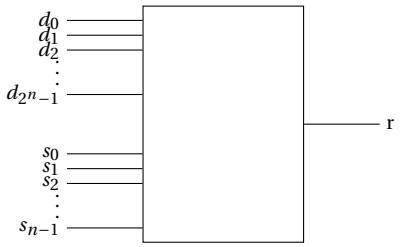


FIGURE 2 – Multiplexeur générique M_n : les données sont sur les entrées d_i, les sélecteurs sont sur les entrées s_i, la sortie en r.

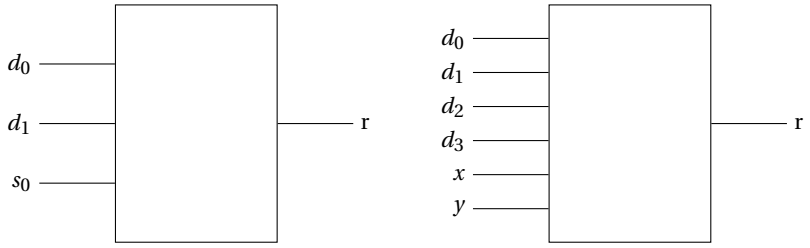
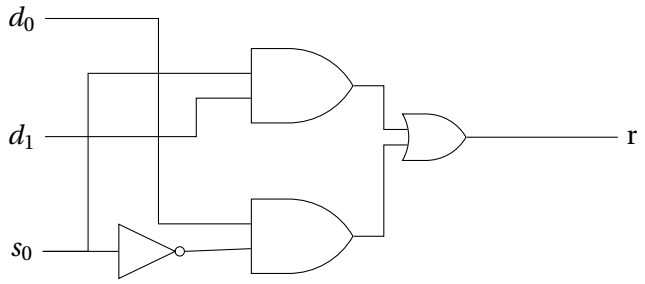


FIGURE 3 – Multiplexeurs : à gauche M_1, cas où $n = 1$, à droite M_2 cas où $n = 2$

B14. En déduire le circuit logique équivalent au multiplexeur pour $n = 1$ à l'aide des portes logiques élémentaires (4 portes en tout). On placera les entrées d_0, d_1 et s_0 à gauche et la sortie r à droite.

Solution : $r = (\neg s_0 \wedge d_0) \vee (s_0 \wedge d_1)$



R Les questions suivantes d'intéressent au multiplexeur M_2 tel qu'il est décrit sur la figure 3. Les entrées de données d_0, d_1, d_2 et d_3 sont supposées **figées** à une certaine valeur 0 ou 1. On cherche alors à **déterminer** la fonction logique $f(x, y)$ calculée par M_2, x et y étant les entrées de sélection.

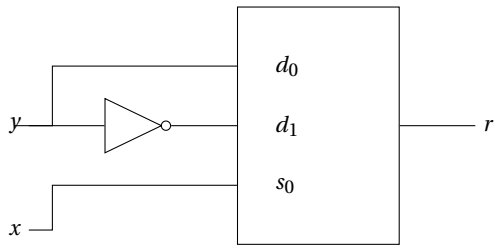
B15. On suppose que $d_0 = 0$, $d_1 = 1$, $d_2 = 1$ et $d_3 = 0$. Construire la table de vérité de la fonction $f(x, y)$ calculée par M_2.

Solution : Les entrées de données permettent de construire directement la table de vérité de la fonction logique f. Il s'agit du ou exclusif : $x \oplus y = (x \wedge \neg y) \vee (\neg x \wedge y)$.

x	y	f(x,y)
0	0	0
0	1	1
1	0	1
1	1	0

B16. Montrer que l'on peut synthétiser la fonction précédente en utilisant M_1, une porte logique et en dupliquant un signal.

Solution : On déduit le circuit de l'équation du ou exclusif $x \oplus y = (x \wedge \neg y) \vee (\neg x \wedge y)$ que l'on met en parallèle de l'équation de M_1 : $r = (\neg s_0 \wedge d_0) \vee (s_0 \wedge d_1)$.

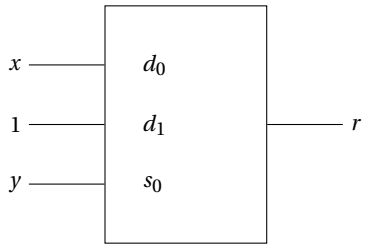


B17. Montrer que l'on peut réaliser la fonction logique $g : \mathscr{B}^2 \to \mathscr{B}$ définie par $g(x, y) = x \vee y$ avec un multiplexeur M_2. Peut-on également la réaliser avec un multiplexeur M_1 et une porte supplémentaire ?

Solution : Pour réaliser g il suffit de fixer les bits de données comme suit : $d_0 = 0$, $d_1 = 1$, $d_2 = 1$ et $d_3 = 1$. La table de vérité est alors celle de $x \vee y$:

x	y	g(x,y)
0	0	0
0	1	1
1	0	1
1	1	1

Il est possible d'utiliser un multiplexeur M_1 comme suit :



B18. Montrer que l'on peut réaliser la fonction logique $h : \mathscr{B}^3 \to \mathscr{B}$ définie par $h(x, y, z) = (x \vee y) \wedge z$ avec un multiplexeur M_3.

Solution : Table de vérité de h et l'on construit M_3 en affectant les valeurs comme indiquées dans la table.

z (s_2)	y(s_1)	x(s_0)	$x \vee y$	h(x,y,z)
0	0	0	0	$0 = d_0$
0	0	1	0	$0 = d_1$
0	1	0	1	$0 = d_2$
0	1	1	1	$0 = d_3$
1	0	0	0	$0 = d_4$
1	0	1	1	$1 = d_5$
1	1	0	1	$1 = d_6$
1	1	1	1	$1 = d_7$

B19. Montrer que l'on peut réaliser la fonction logique quelconque $z : \mathscr{B}^n \to \mathscr{B}$ avec un multiplexeur M_n.

Solution :
D'une manière générale :

- on connecte les variables de la fonction aux entrées de sélection.
- on choisit $d_k = f(x_0, x_1, \ldots, x_{n-1})$ pour $k = \sum_{j=0}^{n-1} 2^j x_j$.

D'une manière générale, on peut construire un multiplexeur M_n à partir de deux multiplexeurs M_{n-1} et d'un M_1 et de duplication de signaux.

B20. On note α_n le nombre de portes logiques nécessaires pour réaliser M_n. En exprimant α_n en fonction de α_{n-1}, déduire une formule explicite de α_n en fonction de n. Conclure !

Solution : On observe que :

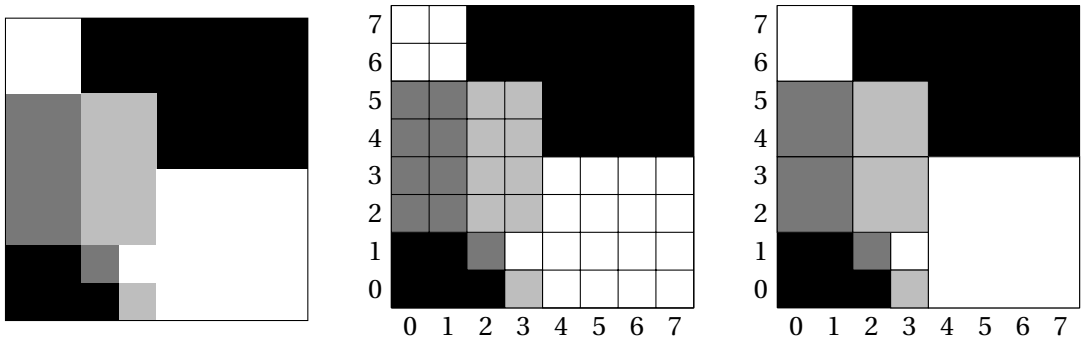
$$\forall n \geqslant 2, \alpha_n = 2\alpha_{n-1} + \alpha_1$$

et $\alpha_1 = 4$. C'est une suite récurrente linéaire d'ordre 1. On calcule le point fixe (-4) et on introduit la suite auxiliaire $\forall n \geqslant 1, \beta_n = \alpha_n + 4$. Finalement, $\beta_n = 2^{n+2}$ et $\alpha_n = 2^{n+2} - 4$.
Pour synthétiser ces circuits, il est donc intéressant de s'appuyer sur des multiplexeurs d'ordre moins élevé, car la croissance du nombre de porte est exponentielle.

C Arbres quaternaires et images (d'après CCINP 2014)

La structure d'arbre quaternaire est une extension de la structure d'arbre binaire qui permet de représenter des informations en deux dimensions. En particulier, la structure d'arbre binaire est utilisée :

- pour représenter de manière plus compacte des images, c'est-à-dire faire de la compression d'image,
- détecter des collisions dans des jeux vidéos,



(1-a) brute (1-b) par pixel (1-c) par bloc

FIGURE 4 – L'image brute (1-a) contient des carrés de différentes couleurs. Cette image est découpée uniformément en pixels (picture elements) dans l'image (1-b). Les pixels sont des carrés de la plus petite taille nécessaire. Ce découpage fait apparaître de nombreux pixels de la même couleur. **Pour réduire la taille du codage**, l'image (1 – c) illustre un découpage dichotomique de l'image en carrés ou **quadrants** qui regroupent certains pixels de la même couleur.

— segmenter des images (classification de zones homogènes).

Un arbre quaternaire décompose une image **par dichotomie sur les deux dimensions jusqu'à obtenir des blocs de la même couleur**. Les images des figures 4 et 5 illustrent ce principe.

R L'objectif de ce problème est l'étude de cette structure d'arbre quaternaire et de son utilisation pour le codage d'images en niveau de gris. **Pour simplifier les programmes, nous nous limitons à des images carrées dont la longueur du côté est une puissance de 2**. Les résultats étudiés se généralisent au cas des images couleurs rectangles de taille quelconque.

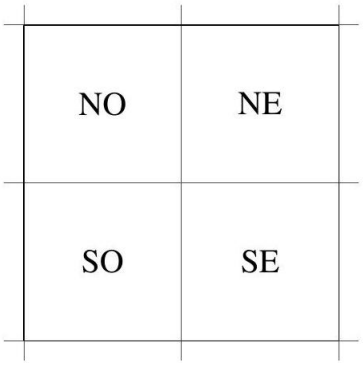
■ **Définition 4 — Arbre quaternaire associé à une image carrée.** Un arbre quaternaire a qui représente une image **carrée** est composé de **nœuds**, qui peuvent être des **blocs** ou des **divisions**. Les ensembles des nœuds, des divisions et des blocs, de l'arbre a sont notés $\mathscr{N}(a), \mathscr{D}(a)$ et $\mathscr{B}(a)$ avec $\mathscr{N}(a) = \mathscr{D}(a) \cup \mathscr{B}(a)$.

Chaque nœud $n \in \mathscr{N}(a)$ contient une abscisse, une ordonnée et une taille, notées $\mathscr{X}(n), \mathscr{Y}(n)$ et $\mathscr{T}(n)$, qui correspondent aux coordonnées et à la longueur du côté de la partie carrée de l'image représentée par n.

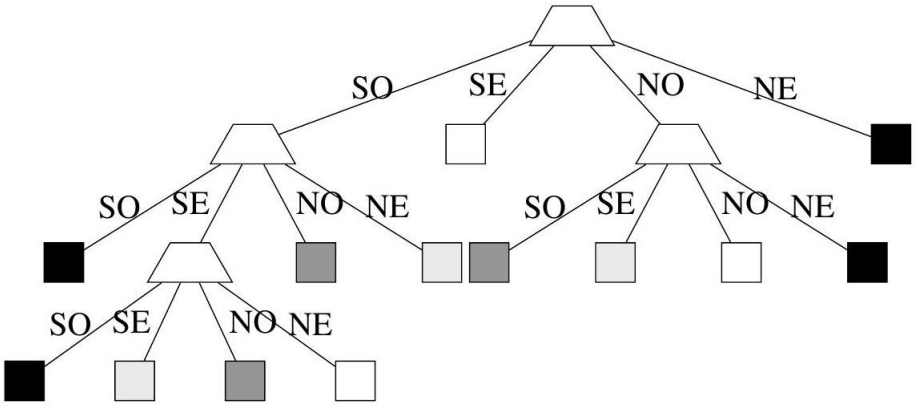
Chaque bloc $b \in \mathscr{B}(a)$ contient une couleur notée $\mathscr{C}(n)$ qui correspond à la couleur de la partie carrée de l'image représentée par b.

Chaque division $d \in \mathscr{D}(a)$ contient quatre fils $f_{SO}, f_{SE}, f_{NO}, f_{NE} \in \mathscr{N}(a)$. Ces fils sont repérés par la position relative de la partie carrée de l'image qu'ils représentent dans l'image représentée par la division d : Sud-Ouest, Sud-Est, Nord-Ouest et Nord-Est.

■ **Exemple 2 — Représentation d'un arbre quaternaire.** La figure 6 contient l'image (3-a) représentée par le sous-arbre situé au Sud-Ouest de l'arbre (2-b) de la figure 5 ainsi que l'indication des coordonnées de chaque point de cette image. La figure 7 contient l'arbre qui représente cette image dont les blocs et les divisions ont été annotés avec les coordonnées, tailles et couleurs.

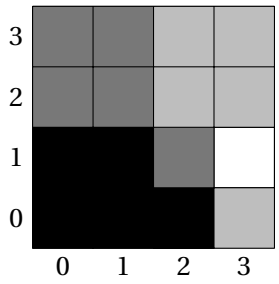


(2-a) position géographique dans une division



(2-b) codage de l'image de la figure 1 par un arbre quaternaire

FIGURE 5 – L'arbre quaternaire de l'image (2-b) représente les carrés contenus dans (1-c). Les étiquettes sur les fils de chaque nœud réprésentent la position géographique des fils dans le nœud : Sud-Ouest, Sud-Est, NordOuest, Nord-Est comme indiqué dans (2-a).



(3-a) Image extraite, représentation matricielle.

FIGURE 6 – Identification des pixels et des carrés (ou quadrants)

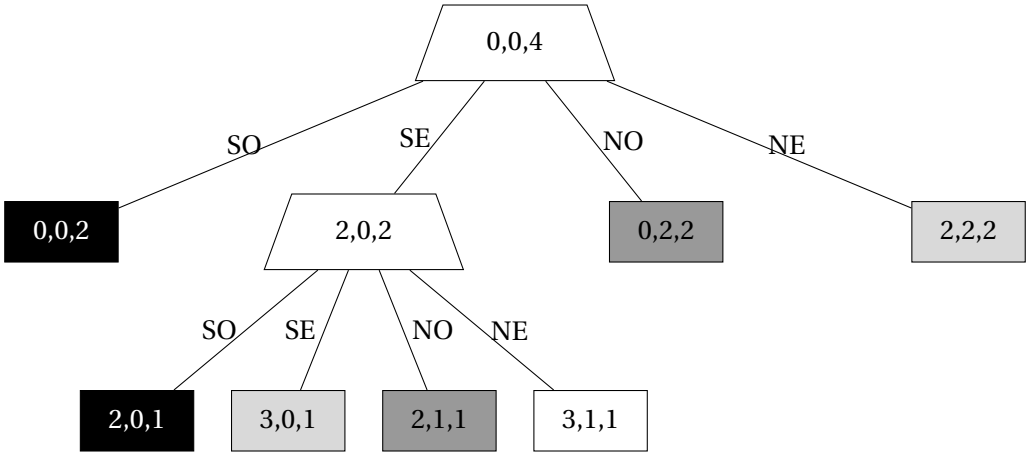


FIGURE 7 – Structure de données correspondant à l'image (3-a)

■ **Définition 5 — Profondeur d'un bloc et hauteur d'un arbre quaternaire.** La profondeur d'un bloc $b \in \mathscr{B}(a)$ d'un arbre quaternaire a est égale au nombre de divisions qui figurent dans la branche conduisant de la racine de a au bloc b. La hauteur d'un arbre a est le maximum des profondeurs de ses blocs $b \in \mathscr{B}(a)$.

■ **Exemple 3 — Hauteur d'un arbre quaternaire.** La hauteur de l'arbre de l'image 2-b est 3. La hauteur de l'arbre de la figure 7 est 2.

C21. Calculer la profondeur maximale que peut atteindre un arbre quaternaire représentant une image $p \times p$ pixels ?

Solution : Soit un arbre quaternaire représentant une image $p \times p$ pixels. Après k subdivisions de l'image, on obtient 4^k divisions de taille $\frac{p}{2^k}$. La profondeur maximale de cet arbre est atteinte lorsque les divisions successives aboutissent à quatre blocs de taille 1 pixel. Dans ce cas, on a $\frac{p}{2^k} = 1$, ce qui peut s'écrire $p = 2^k$. On obtient finalement $k = \log_2 p$. Comme, d'après l'énoncé, p est une puissance de 2, k est un entier.

C22. Donner la prodonfeur maximale d'une image 512x512 pixels.

Solution : Cette profondeur maximale vaut donc $\log_2 512 = \log_2 2^9 = 9$.

Pour construire des arbres quaternaires, on se donne les types OCaml suivants :

```
type quadrant = {
    x : int;
    y : int;
    taille : int;
  }

type couleur = int

type  a4 =
    | Bloc of quadrant * couleur
    | Division of quadrant * a4 * a4 * a4 * a4
```

R La hauteur d'un `Bloc` vaut 0, car il s'agit d'une feuille.

C23. Écrire une fonction de signature `hauteur : a4 -> int` qui calcule la hauteur d'un arbre quaternaire.

Solution :

```
let rec hauteur a =
  match a with
  | Bloc _ -> 0
  | Division (_, so, se, no, ne) ->
      1 + max (max (hauteur so) (hauteur se))  (max (hauteur no) (hauteur ne))
```

C24. Écrire une fonction de signature `nb_blocs : a4 -> int` qui renvoie le nombre de blocs d'un arbre quaternaire.

Solution :

```
let rec nb_blocs arbre =
 match arbre with
  | Bloc (n, c) -> 1
  | Division (_, so, se, no, ne) -> nb_blocs so + nb_blocs se + nb_blocs no + nb_blocs ne
```

Définition 6 — Arbre quaternaire valide. Un arbre quaternaire a est valide, si et seulement si :

1. les tailles, abscisses et ordonnées de chaque nœud $n \in \mathcal{N}(a)$ sont positives,
2. les tailles des quatre fils $f_{SO}, f_{SE}, f_{NO}, f_{NE}$ de chaque division $d \in \mathcal{D}(a)$ sont identiques et égales à la moitié de la taille de d,
3. les abscisses et ordonnées des fils de chaque division $d \in \mathcal{D}(a)$ sont cohérentes avec la position géographique de chaque fils $f_{SO}, f_{SE}, f_{NO}, f_{NE}$ et avec l'abscisse, l'ordonnée et la taille de la division d,
4. si une division $d \in \mathcal{D}(a)$ est composée de quatre éléments de type `Bloc`, alors ces blocs n'ont pas tous la même couleur.

On se dote des fonctions auxiliaires suivantes :

```
let couleur n =
  match n with
  | Bloc (_, c) -> c
  | Division _ -> failwith "Impossible d'obtenir la couleur d'une division"

let quad n = match n with
  | Bloc (q, _) -> q
  | Division (q, _, _, _, _) -> q
```

C25. Écrire une fonction de signature `coord_pos : a4 -> bool` qui statue sur le respect de la propriété 1 de la définition 6. Le paramètre `a4` de la fonction est considéré comme un nœud sur lequel on tente de vérifier la propriété, c'est-à-dire on vérifie la propriété sur le nœud uniquement, pas sur tout l'arbre.

Solution :

```
let coord_pos n = let q = quad n in q.taille >= 0 && q.x >= 0 && q.y >= 0;
```

R L'ordre des arbres quaternaires dans un type `Division` est celui donné à la défintion 4, c'est-à-dire Sud-Ouest, Sud-Est, Nord-Ouest et Nord-Est.

C26. Écrire une fonction de signature `verifier_pos : a4 -> bool` qui statue sur le respect des propriétés 1, 2 et 3 de la définition 6 par le nœud passé en paramètre. On s'appuiera sur la fonction précédente pour 1 et sur une fonction auxiliaire pour vérifier 2 et 3 sur le nœud `a4`.

Solution :

```
let verifier_pos n =
     let check q so se no ne = let demi_taille = q.taille / 2 in
        so.x = q.x && so.y = q.y && so.taille = demi_taille &&
        se.x = q.x + demi_taille && se.y = q.y && se.taille = demi_taille &&
        no.x = q.x && no.y = q.y + demi_taille && no.taille = demi_taille &&
        ne.x = q.x + demi_taille && ne.y = q.y + demi_taille && ne.taille = demi_taille in
    match n with
    | Bloc (_,_) -> coord_pos n
    | Division (q, so, se, no, ne) ->  coord_pos n && check q (quad so) (quad se) (quad no
        ) (quad ne)
```

C27. Écrire une fonction de signature `verifier_couleur : a4 -> bool` statue sur le respect de la propriété 4 de la définition 6 par le nœud passé en paramètre.

Solution :

```
  let verifier_couleur d =
    match d with
    | Division (q,Bloc(_,cso),Bloc(_,cse),Bloc(_,cno),Bloc(_,cne))  ->
            not (cso = cse && cse = cno && cno = cne)
    | _ -> true
```

C28. Écrire une fonction **récursive** de signature `vaq : a4 -> bool` qui vérifie qu'un arbre quaternaire est valide.

Solution :

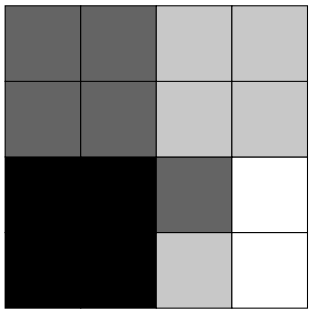
```
  let rec vaq arbre =
    match arbre with
    | Bloc (n, _) -> verifier_pos arbre
    | Division (_, so, se, no, ne) -> verifier_pos arbre && verifier_couleur arbre
                                      && vaq so && vaq se && vaq no && vaq ne
```

Une image est représentée par un type `int array array`. Le pixel de coordonnées (x, y) se trouve dans la case (x, y). L'intensité de chaque pixel est codé de 0 (noir) à 255 (blanc). On se donne l'image suivante :

```
let img = [|
  [|0; 0; 200; 255|];
  [|0; 0; 100; 255|];
  [|100; 100; 200; 200|];
  [|100; 100; 200; 200|]
  |]
```

C29. Dessiner l'image `img` en suivant le modèle 3-a

Solution :



C30. Écrire une fonction de signature `couleur_uniforme : couleur array array -> int -> int -> int -> bool` qui prend comme paramètres une image `img` et trois entiers `ligne,colonne,taille`. Cette fonction statue sur le fait que la sous-matrice carrée de l'image correspondant au quadrant `(ligne, colonne,taille)` est d'une couleur uniforme. Ainsi, l'appel `couleur_uniforme img 0 0 2` renvoie `true` et `couleur_uniforme img 0 2 2` renvoie `false`.

Solution :

```
let couleur_uniforme matrice ligne colonne taille =
let c = matrice.(ligne).(colonne) in
let uniforme = ref true in
let i = ref ligne in
let j = ref colonne in
while !uniforme && !i < ligne + taille do
    if matrice.(!i).(!j) <> c then uniforme := false;
    incr j;
    if !j  = colonne + taille && !i < ligne + taille then (incr i; j := colonne);
done;
!uniforme;;
```

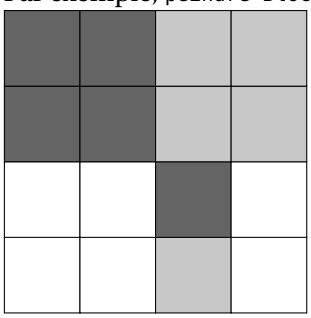
C31. Écrire une fonction de signature `img_vers_arbre : couleur array array -> a4` qui construit un arbre quaternaire à partir de la matrice d'une image. S'appuyer sur la fontion précédente et utiliser une fonction auxiliaire récursive qui découpe l'image selon le besoin.

Solution :

```
let img_vers_arbre m =
 let rec aux ligne colonne taille =
  if couleur_uniforme m ligne colonne taille
  then let c = m.(ligne).(colonne)
           in Bloc ({x = ligne; y = colonne; taille = taille}, c)
  else (
    let demi = taille / 2 in
    let so = aux ligne colonne demi in
    let se = aux ligne (colonne + demi) demi in
    let no = aux (ligne + demi) colonne demi in
    let ne = aux (ligne + demi) (colonne + demi) demi in
    Division ({x=ligne; y=colonne; taille}, so, se, no, ne)
  ) in
```

```
let taille = Array.length m in aux 0 0 taille
```

C32. Écrire une fonction de signature `peindre : Bloc -> couleur array array -> unit` qui modifie sur l'image passée en paramètre les pixels du bloc conformément à la couleur du bloc.
Par exemple, `peindre Bloc({x=0,y=0,taille=2},255)img` va transformer l'image exemple en :



Solution :

```
let peindre b img =
  let c = couleur b in
  let q = quad b in
  for i = q.x to q.x + q.taille - 1 do
     for j = q.y to q.y + q.taille - 1 do
        img.(i).(j) <- c
     done;
  done
```

C33. Écrire une fonction de signature `arbre_vers_img : a4 -> int array array` qui transforme un arbre quaternaire en une image sous la forme d'une matrice.

Solution :

```
let arbre_vers_img a =
    let q = quad a in
    let img = Array.make_matrix q.taille q.taille 0 in
    let rec pblocs t =
       match t with
        | Bloc(q,c) -> peindre t img
        | Division (q,so,se,no,ne) -> pblocs so; pblocs se; pblocs no; pblocs ne;
    in pblocs a;
    img
```

R Dans le domaine de la compression de l'information, on distingue des méthodes de compression avec pertes et sans pertes. On dit qu'une méthode est avec pertes, si l'information a été dégradée d'une manière ou d'une autre au cours de processus de compression/décompression.

C34. Citer une méthode de compression d'information sans pertes et décrire sommairement son principe.

> **Solution :** Le codage d'Huffmann est une méthode de compression d'information sans pertes.

C35. La compression d'une image avec un arbre quaternaire est-elle sans perte ? Donner le principe de la démonstration qui pourrait vous permettre de le démontrer.

> **Solution :** Oui, car on peut reconstituer l'image sans aucune altération d'après l'arbre. L'encodage sous la forme d'un arbre quatenaire ne perd aucune information sur aucun pixel.
> On peut le démontrer par induction structurelle sur les arbres quaternaires.

C36. Il faut un octet pour encoder un pixel dans une matrice, 8 octets pour encoder un type `Bloc` et 24 octets pour un type `Division`. Pour quel type d'image la compression d'une image sous la forme d'un arbre quaternaire peut-t-elle être efficace ?

> **Solution :** La compression par un arbre quaternaire ne sera pas efficace sur une image bruitée ou sur une image très détaillée. Par contre, sur une image qui possède de nombreuses zones homogènes délimitées, comme une carte ou une modélisation numérique, cette compression sera très efficace. D'où son utlisation pour les modèle numériques (logiciels de modélisation physique et mécanique, jeux vidéos, ...)

D Bonus fractal

Soit a et b deux nombres réels. On définit deux suites récurrentes $(x_n)_{n\in\mathbb{N}}$ et $(y_n)_{n\in\mathbb{N}}$ par :

$$x_0 = 0 \tag{1}$$
$$x_{n+1} = x_n^2 - y_n^2 + a \tag{2}$$
$$y_0 = 0 \tag{3}$$
$$y_{n+1} = 2x_n y_n + b \tag{4}$$
$$\tag{5}$$

L'ensemble de Mandelbrot est l'ensemble de points (a,b) du plan pour lesquels aucune des deux suites récurrentes ne tend vers l'infini en valeur absolue. On peut démontrer que $(x_n)_{n\in\mathbb{N}}$ ou $(y_n)_{n\in\mathbb{N}}$ tend vers l'infini dès que $x_n^2 + y_n^2 > 4$. On en déduit que le disque de rayon deux et centré à l'origine du plan (0,0) contient tous les points de l'ensemble de Mandelbrot.

On ne sait pas caractériser autrement la divergence de ces suites. C'est pourquoi, afin de tracer une approximation de l'ensemble de Mandelbrot, on se contentera de dire qu'un point appartient à l'ensemble si, pour $K \in \mathbb{N}$ fixé, la condition

$$\forall i \in \mathbb{N}, i \leqslant K, x_i^2 + y_i^2 \leqslant 4 \tag{6}$$

est respectée[2].

Au-delà d'un certain rang K, si on n'a pas la preuve que les suites divergent, on fera l'approximation qu'elles sont convergentes et que le point (a, b) appartient à l'ensemble.

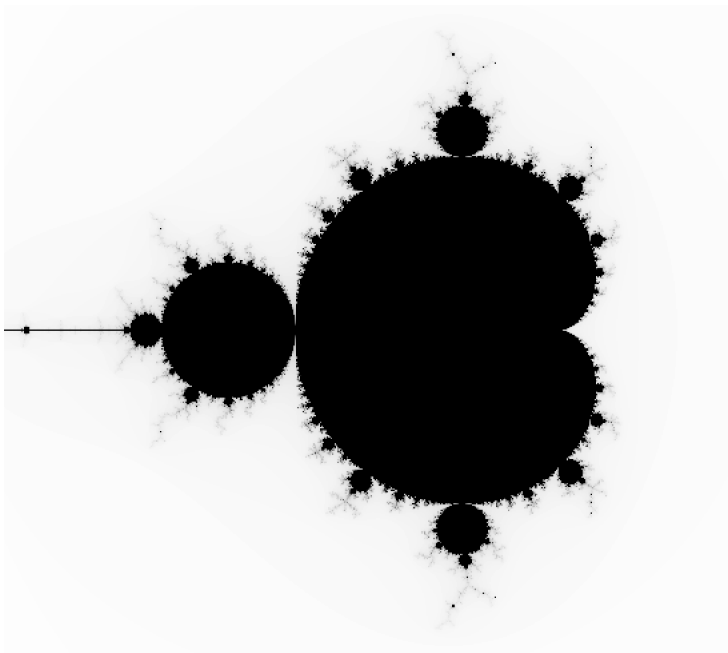


FIGURE 8 – Représentation de l'ensemble de Mandelbrot

La figure 8 représente l'ensemble de Mandelbrot : plus les points sont clairs, plus la convergence du test d'appartenance à l'ensemble est longue. Les fonctions programmées dans cette section permettent de reproduire cette figure.

D37. Écrire une fonction de signature `norme_carree : float -> float -> float` qui renvoie la norme euclidienne au carré du vecteur de coordonnées (x, y), c'est à dire $x^2 + y^2$. Les deux coordonnées `x` et `y` sont transmises en paramètres à la fonction.

Solution :

```
1  let norme_carree x y = x *. x +. y *. y
```

D38. Écrire une fonction de signature `mandelbrot : float -> float -> int -> int` dont les paramètres sont les cordonnées a et b d'un point du plan de type `float` et la limite K. Cette fonction renvoie le rang n des suites x et y nécessaires pour vérifier si le point (a, b) appartient à l'ensemble de Mandelbrot sous la limite K.

Solution :

```
1  let mandelbrot a b k =
```

2. C'est ainsi qu'on a obtenu la première image de cet ensemble.

```
2     let rec calculer n x y =
3       if n = k || norme_carree x y > 4. then n
4       else calculer (n+1) (x *. x -. y *. y +. a)  (2. *. x *. y +. b)
5     in
6     calculer 0 0.0 0.0
```

D39. Démontrer la terminaison de la fonction précédente.

Solution : La suite des paramètres des appels récursifs est $u_{i+1} = u_i + 1$ et $u_0 = 0$. C'est une suite arithmétique strictement croissante. Nécessairement, la condition d'arrêt est atteinte au bout de k itérations.

D40. Quelle est la complexité de la fonction `mandelbrot` dans le pire des cas ?

Solution : Dans le pire des cas, le nombre d'apples récursifs est k. Posons $i = k - n$:

$$T(i) = 1 + T(i-1)$$

équation récurrente caractéristique d'une complexité linéaire en i. Pour le démontrer, on peut procéder par téléscopage ou en remarquant qu'il s'agit d'une suite arithmétique de raison 1 dont on connaît la formulation explicite. C'est pourquoi, la complexité dans le pire des cas de la fonction est linéaire en $O(i) = O(k-n) = O(k)$.